

CSCE 606 SOFTWARE ENGINEERING · MODULE 02 · LECTURE NOTES

Requirements and Specification

Finding out what to build, and writing it down checkably.

MODULE OUTCOMES

Explain why requirements elicitation is difficult rather than lazy.

Distinguish functional from quality requirements.

Write a specification precise enough to be checked.

Use preconditions, postconditions, and invariants.

Recognise when a requirement is actually a proposed solution.

1 Why requirements are hard

The difficulty is structural rather than a matter of people being unhelpful or analysts being lazy.

- 1 **People do not know what they want until they see something.** Preferences are constructed on contact with a concrete artefact, not retrieved from storage. This is why prototypes elicit better requirements than interviews.
- 2 **People describe solutions rather than problems.** A request arrives already shaped as a design, and the problem that motivated it is left unstated.
- 3 **The important knowledge is tacit.** Experts cannot easily articulate what they know, and the constraints that matter most are the ones so obvious to everyone in the domain that nobody thinks to mention them. These are the requirements that are discovered at acceptance testing.
- 4 **Stakeholders conflict.** Different people want incompatible things and frequently do not know it, because they have never had to state their assumptions side by side.
- 5 **Requirements change** — because the business changes, because competitors change, and because deploying your system changes what people do and therefore what they need.

THE SINGLE MOST USEFUL TECHNIQUE: ASK WHY

'We need a dropdown to select the warehouse' is not a requirement. It is a design, proposed by someone who diagnosed a problem they did not state.

Ask why. Perhaps orders are shipping from the wrong warehouse. *That* is the requirement, and once it is visible, a dropdown may be among the worse solutions — the system could infer the warehouse from the delivery address, or validate the choice afterwards, or make the wrong selection structurally impossible.

Keep asking until you reach something the stakeholder values independently of any particular implementation. That is the requirement; everything above it is negotiable, and the negotiation is where engineering adds value.

2 Two kinds of requirement

	Functional	Quality (non-functional)
Describes	What the system does.	How well it does it.
Example	'A user can reset their password via an emailed link.'	'The reset email arrives within 30 seconds for 99% of requests.'
Stated?	Usually, and explicitly.	Rarely, because they are assumed.
Testable?	Generally straightforward.	Requires load, chaos, or security testing to verify.
Retrofittable?	Usually, at the cost of a feature's work.	Often not. They constrain the architecture.

QUALITY REQUIREMENTS DETERMINE THE ARCHITECTURE

A functional requirement discovered late typically costs the work of building that feature. A quality requirement discovered late can cost a rewrite.

'It must sustain a thousand times current load', 'every change must be auditable', 'it must work offline', 'no customer data may leave the region' — none of these are features. Each is a constraint that determines how the system is structured, and retrofitting one means restructuring.

They are also exactly the requirements nobody volunteers, because they feel too obvious to say. Nobody says 'the system should not lose data'.

So ask explicitly and in numbers. How fast? Measured how, at what percentile, under what load? How available, and what does downtime cost? How many users in year three? What must happen when a dependency fails? Who must be prevented from seeing what?

3 Writing a specification

A specification is a claim about the system that can turn out to be false. If it cannot be false, it is not a specification.

Property	Meaning	Failure mode
Checkable	You can determine whether the system satisfies it.	'The system shall be fast' — nobody can tell whether it complies. '95th-percentile response under 200 ms at 1,000 requests per second' — anyone can.
Complete enough	Covers the situations that will arise, including failures.	Specifying the happy path only, so error behaviour is invented independently by each implementer.
Unambiguous	Two competent readers reach the same conclusion.	'The system should handle invalid input gracefully' — handle how?
Minimal	Constrains what matters; leaves the rest free.	Over-specification. Mandating an implementation detail forbids improvements nobody intended to forbid, and the constraint outlives the reason for it.

3.1 Contracts

```
def withdraw(account, amount):
    """Withdraw `amount` from `account`.

    PRECONDITION
        amount > 0 and amount <= account.balance
        (the CALLER must ensure this)

    POSTCONDITION
        account.balance is reduced by exactly `amount`;
        a transaction record is appended
        (the IMPLEMENTATION must ensure this)

    INVARIANT
        account.balance == sum(t.delta for t in account.transactions)
        (true before and after; may be temporarily false inside)
    """
```

THE POINT OF A CONTRACT IS ASSIGNING RESPONSIBILITY

If the precondition is violated, the bug is in the **caller**. If the postcondition fails while the precondition held, the bug is in the **implementation**. The contract makes that determination mechanical rather than a matter of argument.

Without contracts, every function defensively validates everything it receives, nobody is sure whose responsibility anything is, and a large fraction of the codebase becomes redundant checking — which is itself a source of bugs, because the checks disagree.

STRONG PRECONDITIONS OR DEFENSIVE VALIDATION?

A strong precondition places the obligation on the caller. The function is simpler and faster, and a violation is undefined behaviour — potentially a security problem.

Defensive validation checks and raises. Safer, and every caller pays the cost, and every caller must now handle an error that may be impossible in their situation.

The practical resolution is to validate at system boundaries. Data arriving from a user, a network, a file, or another service is untrusted and must be checked once, thoroughly, at the point of entry. Within the system, rely on preconditions — and assert them in debug builds, which gives you the checking during development and the speed in production.

Validating the same property at every layer is how codebases end up mostly validation, and it provides less safety than it appears to, because the layers eventually disagree about what is valid.

4 Formats

Format	Strength	Weakness
User story	Keeps the user's goal visible and resists premature design.	Imprecise on its own. Useless without acceptance criteria.
Use case	Documents a complete flow including alternatives and error paths.	Verbose; falls out of date.
Acceptance criteria	Checkable conditions. The part that does the work.	Can lose sight of why the feature exists.
Executable specification (Cucumber, BDD)	The specification and the test are one artefact, so they cannot diverge.	Substantial tooling overhead; the natural-language layer often adds cost without adding readers.
Formal specification (TLA+, Alloy)	Can prove properties of concurrent and distributed protocols that testing cannot reach.	Expensive and requires specialist skill. Worth it for a protocol whose failure is catastrophic — AWS uses TLA+ on core services and reports finding bugs that had survived extensive testing.

```
As a warehouse manager
I want to see which orders are delayed
So that I can contact customers before they contact me
```

ACCEPTANCE CRITERIA

- "Delayed" = promised date in the past AND status != Shipped.
- Sorted by lateness, most late first.
- Orders less than 1 hour late are excluded.
- Empty result shows "No delayed orders", not a blank table.
- Page loads in under 2 seconds with 10,000 open orders.

The **so that** clause carries the actual requirement. Knowing that the goal is contacting customers before they complain opens options the original request did not: perhaps the system should notify customers automatically, which serves the goal better than any list.

The last two criteria are the ones habitually omitted. Empty-state behaviour and performance under realistic data volume are requirements, and discovering them after release is how a shipped feature becomes unusable.

5 Requirements change

CHANGE IS NOT AN ELICITATION FAILURE

The instinct is to treat changing requirements as evidence that the analysis was inadequate. Usually it is not. The business environment changes, competitors act, regulations shift — and deploying the system changes what users do, which changes what they need next.

So the goal is not to make requirements complete and final before building. It is to build something that **absorbs change cheaply**, which is a design property (Module 03) rather than a process one.

Three things follow. **Deliver early**, so feedback arrives while acting on it is still cheap. **Keep modules loosely coupled**, so a change in requirements touches one place. **Record decisions and their reasons** — an architecture decision record costs ten minutes to write and saves re-deriving the argument a year later, usually in a meeting.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
MIT 6.031 — Specifications, Designing Specifications	The most precise free treatment of preconditions, postconditions, and specification strength. Read both readings.
Software Engineering at Google — Chapter 1 (and the 'what do we mean' framing)	On requirements that are discovered rather than stated.
Newcombe et al. — How Amazon Web Services Uses Formal Methods (free)	An honest industrial account of where formal specification pays and where it does not.
Michael Nygard — Documenting Architecture Decisions	The ADR format. Two pages, and widely adopted for good reason.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Take a feature request phrased as a solution — from your own project, or an open-source issue tracker — and work backwards to the underlying problem by asking why repeatedly. Propose two alternative solutions.
- 2 Write acceptance criteria for a feature you have built. Include the empty state, the error cases, and a performance criterion. Then check your implementation against them and record what fails.
- 3 Write preconditions, postconditions, and an invariant for three functions in an existing codebase. Assert them in a debug build and run the test suite. Report any that fire.

- 4 Take a requirement written as 'the system shall be fast/secure/reliable' and rewrite it so that compliance can be determined.
- 5 Find a quality requirement in a system you use that was clearly discovered late — bolted-on auditing, a pagination system added under duress. Describe the architectural damage.
- 6 Write an architecture decision record for a decision you have already made, including the alternatives you rejected and why.
- 7 For your Project 2 open-source target, read three closed issues and classify each as a well-specified requirement or a proposed solution.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Give five structural reasons requirements elicitation is difficult.
- 2 Why is a stated requirement usually a proposed solution, and what do you do about it?
- 3 Distinguish functional from quality requirements, and say why the latter are harder to retrofit.
- 4 Name the four properties of a useful specification, and give the failure mode of ignoring minimality.
- 5 What does a contract accomplish beyond documentation?
- 6 Where should you validate inputs, and what goes wrong if you validate everywhere?
- 7 Why is changing requirements not a failure of analysis, and what does that imply for design?